An algebraic two-level Schwarz preconditioner

Design, performance studies and comparison with OpenFOAM GAMG on the pressure equation of bubbly flows

Francesco Buffoli

October 2026

Advanced Programming for Scientific Computing - Prof. L. Formaggia
Politecnico di Milano

Index

Fully Algebraic Two-level Schwarz

1. Introduction
2. Code structure
3. Partition
4. One-level Schwarz preconditioner
5. Lumping & local block splitting
6. Coarse space: eigenvalue problem
7. Solvers
8. Reproduction of the paper

The Pressure Equation of Bubbly Flows

9. Bubble flow: problem, setup reuse, cheaper solves
10. Performance vs GAMG
11. Conclusions

Reference: H. Al Daas, P. Jolivet, T. Rees, *Efficient algebraic two-level Schwarz preconditioner for sparse matrices*, 2023.

Code: `github.com/fraBuffoli/Algebraic-Multilevel-Preconditioner`

1. Introduction: One-Level Schwarz

Owners. METIS splits the graph $\mathcal{G}(A + A^\top)$ into N disjoint sets $\Omega_{I,i}$, one per process: a subdomain is a set of rows of A, no mesh is needed.

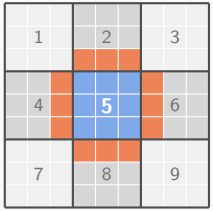
Overlap. Each set is extended with its neighbours at distance one, $\Omega_{\Gamma,i}$, the unknowns its rows are coupled to: $\Omega_i = [\Omega_{I,i}, \Omega_{\Gamma,i}]$.

Local problems. $R_i = I_n(\Omega_i, :)$, $A_{ii} = R_i A R_i^\top$; the diagonal matrices D_i form a partition of unity, $\sum_i R_i^\top D_i R_i = I_n$.

$$M_{\mathrm{ASM}}^{-1} = \sum_{i=1}^{N} R_i^\top A_{ii}^{-1} R_i$$

$$M_{\mathrm{RAS}}^{-1} = \sum_{i=1}^{N} R_i^\top D_i A_{ii}^{-1} R_i$$

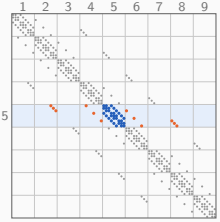
Why one level is not enough. At each application information only reaches the neighbouring subdomains: a smooth error needs $\sim\sqrt{N}$ iterations to cross the domain (2D) $\Rightarrow$ a coarse correction is needed.



$\Omega_{I,5}$

$\Omega_{\Gamma,5}$

the other 8 owners



owned by 5: its `LocalSystem`

$A(\Omega_{I,5}, \Omega_{I,5})$

$A(\Omega_{I,5}, \Omega_{\Gamma,5})$

1. Introduction: Coarse Correction

Given $R_0 \in \mathbb{R}^{n_0 \times n}$ whose rows span a coarse space of dimension $n_0 \ll n$, and $A_{00} = R_0 A R_0^\top$:

$$M^{-1}_{\star,\text{additive}} = R_0^\top A_{00}^{-1} R_0 + M_\star^{-1},$$
$$M^{-1}_{\star,\text{deflated}} = R_0^\top A_{00}^{-1} R_0 + M_\star^{-1}\left(I - A R_0^\top A_{00}^{-1} R_0\right), \qquad \star \in \{\text{ASM}, \text{RAS}\}.$$

Deflated: coarse correction first, then the local solves on the deflated residual $r - Ax_0$, $x_0 = R_0^\top A_{00}^{-1} R_0 r$: one extra product with A.

Default here, as in the experiments of the paper: RAS + deflated.

Key idea of the paper

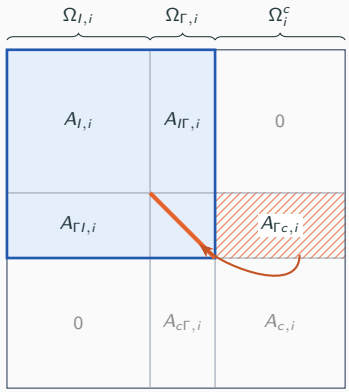
R_0 is built algebraically, from local generalized eigenproblems that only use entries of A: A_{ii}, D_i and a lumped splitting $\widetilde{A}_{ii}$. No mesh, no near-kernel vectors, no Neumann matrices from a finite element assembly.

1. Introduction: Local Block Splitting

Lumping in the overlap. Local ordering $[\Omega_{I,i}, \Omega_{\Gamma,i}, \Omega_i^c]$: the absolute row sums s_i of $A(\Omega_{\Gamma,i}, \Omega_i^c)$, i.e. the couplings of the overlap with the outside, are taken off the diagonal of the overlap block:

$$\widetilde{A}_{ii} = \begin{pmatrix} A_{I,i} & A_{I\Gamma,i} \\ A_{\Gamma I,i} & A_{\Gamma,i} - \mathrm{diag}(s_i) \end{pmatrix}$$

Only the diagonal of the overlap block changes: $O(n_i)$, no communication once the overlap rows are known.



A_{ii}; $\widetilde{A}_{ii}$ is the same block, except the orange diagonal

$A_{\Gamma c,i}$: its absolute row sums give s_i

diagonal of $A_{\Gamma,i}$: $-\mathrm{diag}(s_i)$

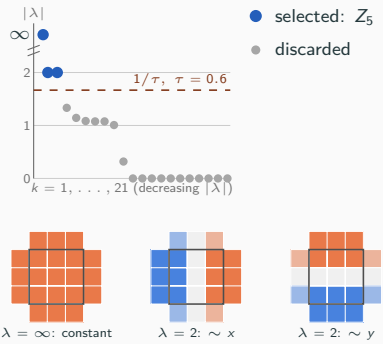
1. Introduction: Coarse Space

Local eigenproblem. $B_i = D_i A_{ii} D_i$, $L_i = \ker B_i$, $K_i = \ker \widetilde{A}_{ii}$, Π_i projection onto $\mathrm{range}(\widetilde{A}_{ii})$. Find $(\lambda, u) \in \mathbb{C} \times \mathbb{C}^{n_i}$ s.t.: $\Pi_i B_i \Pi_i \, u = \lambda \, \widetilde{A}_{ii} \, u$

Coarse space. Z_i spans

$$\underbrace{(L_i \cap K_i)^{\perp_{K_i}}}_{\text{infinite eigenvalues}} \oplus \mathrm{span}\{u : \ |\lambda| \geq 1/\tau\}$$

Modes with high $|\lambda|$ are selected: the local correction does not effectively reduce their error components; represented in the coarse space: $R_0^\top = [R_1^\top D_1 Z_1, \ldots, R_N^\top D_N Z_N]$

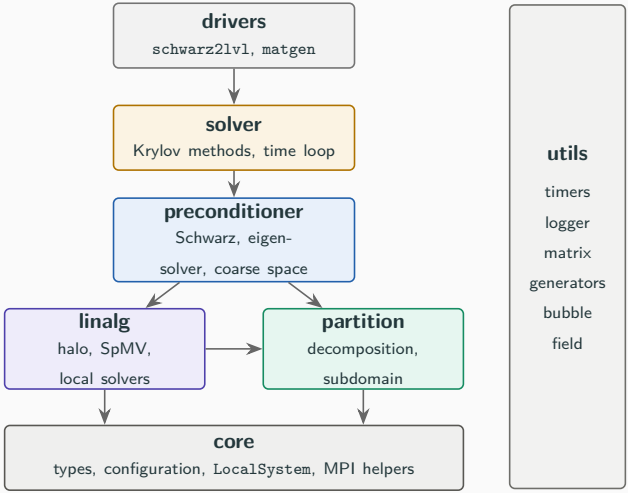


Bound for $M^{-1}_{\mathrm{ASM,add}}$, SPD and diag. dominant A

$$\kappa\left(M^{-1}A\right) \leq (k_c + 1)\left(2 + (2k_c + 1)\, k_m/\tau\right)$$

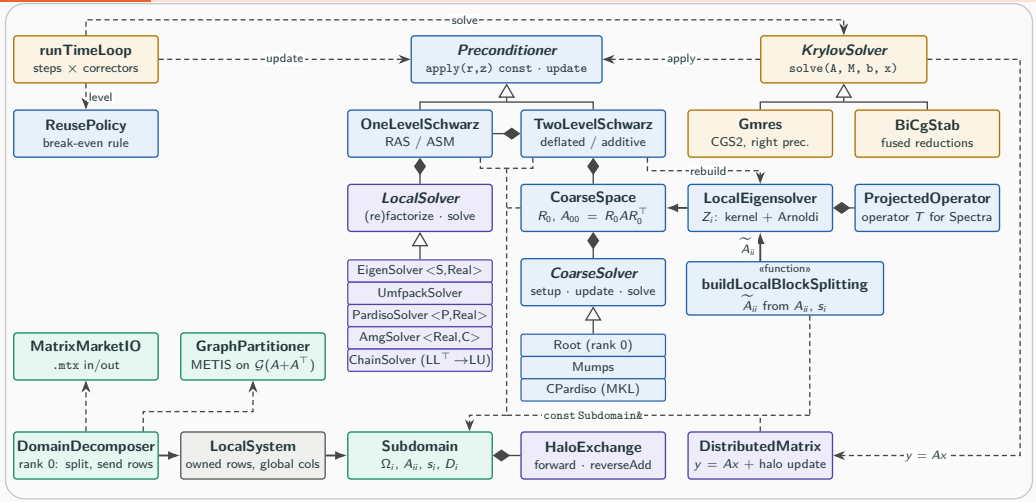
k_c: colours of the subdomain graph, k_m: max subdomains per row; independent of N and of the coefficients.

2. Code structure: modules



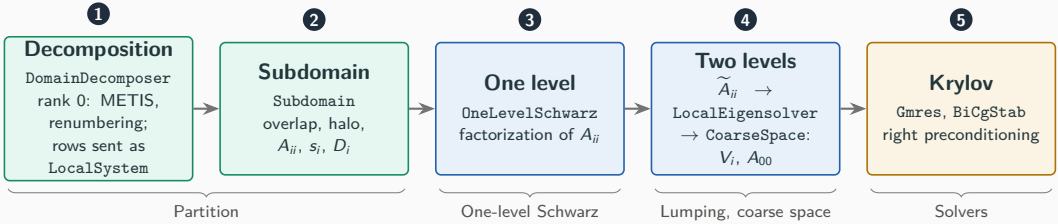
The colours of the modules are kept in every class diagram that follows.

2. Code structure: main classes



italic: abstract interface —▷ inheritance —◆ owns (`unique_ptr`) ----→ uses —→ data passed on

2. Code structure: flow of a solve



After step 1 no process holds global data: every global operation is a reduction, a neighbour exchange, or the assembly of A_{00}.

One MPI process = one subdomain. Every phase is timed (`ScopedTimer`: min / avg / max over the ranks).

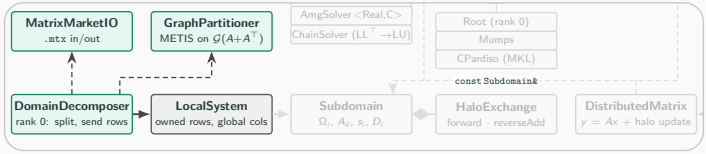
3. Partition: decomposition

`DomainDecomposer::decompose`, as OpenFOAM `decomposePar`: all global work on rank 0, once.

1. Read a Matrix Market file or run a generator.
2. METIS k-way on $\mathcal{G}(A + A^\top)$, fixed seed.
3. Renumber: rank p owns the contiguous range $[o_p, o_{p+1})$.
4. Send every rank only its rows (CSR, global columns).

```
struct LocalSystem { // per rank
  MPI_Comm comm;
  GlobalIndex n_global;
  std::vector<GlobalIndex>
  ownership, // [o_p, o_p+1)
  row_ptr, col; // owned rows
  std::vector<Scalar> val, rhs;
  int  block_size; // per cell
  bool symmetric;
};
```

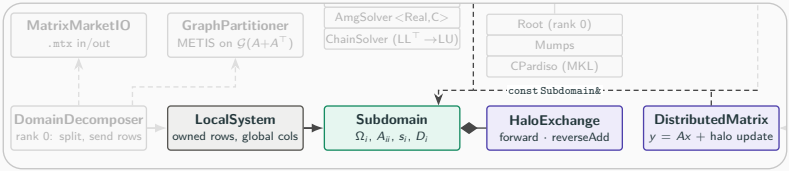
Result: `LocalSystem` = what an OpenFOAM processor owns: `lduMatrix` + `globalIndex`.



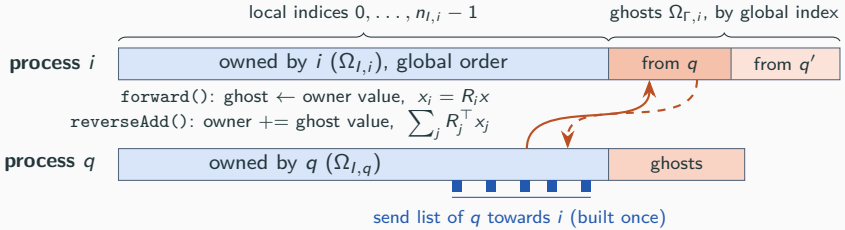
3. Partition: subdomain

`Subdomain(const LocalSystem&, pou)`: each rank builds its own overlapping subdomain, collectively, without global data.

1. **Overlap** $\Omega_{\Gamma,i}$: off-process columns of the owned rows, plus remote rows with a column in $\Omega_{I,i}$ (one `Alltoall` + one `Alltoallv`).
2. **Halo** pattern → `HaloExchange`.
3. **Ghost rows** $A(\Omega_{\Gamma,i}, :)$ from their owners.
4. **Local matrices** A_{ii} (col-major) and $A(\Omega_{I,i}, \Omega_i)$ (row-major); lumping sums s_i while ghost rows arrive.
5. **Partition of unity**: Default is `bool`, $D_i = 1$ on $\Omega_{I,i}$, 0 on $\Omega_{\Gamma,i}$.



3. Partition: halo exchange and distributed product



Contiguous ownership: the ghosts of one neighbour are a contiguous block, received in place (`MPI_Irecv` into the vector).

Non-blocking point-to-point, neighbours only; buffers reused.

`exchange<T>()` for irregular data (ghost rows, sparse rows).

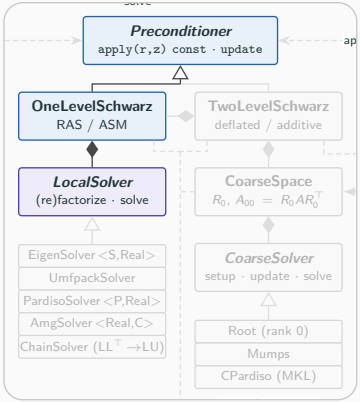
```
// DistributedMatrix::apply = lduMatrix::Amul
void apply(const Vec& x, Vec& y) const {
  xloc_.head(sd_.nOwned()) = x;
  sd_.halo().forward(xloc_); // ghost values
  y.noalias() = sd_.Aowned() * xloc_;
}
```

4. One-level Schwarz

```
void OneLevelSchwarz::apply(const Vec& r, Vec& z) const {
  rloc_.head(nI) = r;
  sd_.halo().forward(rloc_);           // R_i r
  solver_->solve(rloc_, yloc_);        // A_ii^{-1} R_i r
  if (variant_ == Variant::RAS) {
    yloc_.array() *= sd_.pou().array(); // D_i ...
    if (!sd_.booleanPou()) sd_.halo().reverseAdd(yloc_);
  } else
    sd_.halo().reverseAdd(yloc_);      // sum_j R_j^T ...
  z = yloc_.head(nI);
}
```

Vectors hold owned entries only; the overlap lives in work vectors.

Boolean PoU (default): RAS needs one `forward()`, no reverse communication.



4. One-level Schwarz: local solvers

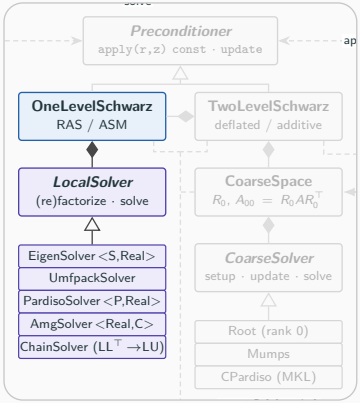
Strategy interface `LocalSolver`: `factorize`, `refactorize` (symbolic analysis reused), multi-RHS `solve`, `solveTranspose`.

Backends chosen at run time: Eigen (SparseLU, LLT, LDLT), UMFPACK, MKL PARDISO, one AMGCL V-cycle.

Built by a factory, `makeLocalSolver(spec)`: backend, factorization, symmetry, precision.

`ChainSolver`: Cholesky $\to$ LDL$^\top$ $\to$ LU, keeps the first that works (symmetric only if A is *known* to be symmetric).

Precision as a template parameter; `if constexpr` removes the conversions in the `double` instantiation.

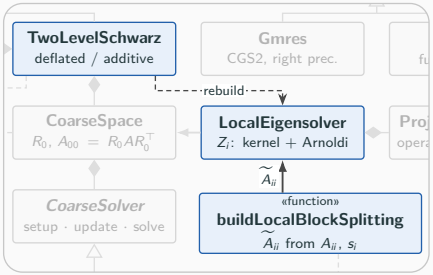


On G100

PARDISO runs 2 steps of iterative refinement per solve: disabled $\Rightarrow$ **3–3.7**$\times$ faster, same iterations.

5. Lumping & local block splitting

```
SpMat buildLocalBlockSplitting(const Subdomain& sd) {
  SpMat At = sd.Aii();
  const Vec& s = sd.lumpingSums();  // from Subdomain
  for (LocalIndex j = 0; j < sd.nGhost(); ++j) {
    const LocalIndex l = sd.nOwned() + j;
    At.coeffRef(l, l) -= s[j]; // A_Gamma - diag(s)
  }
  At.makeCompressed();
  return At;
}
```



$s_i(j) = \sum_{k \notin \Omega_i} |A(j,k)|$: summed by `Subdomain`; the outside of Ω_i is never stored.

Consequence for the eigensolver

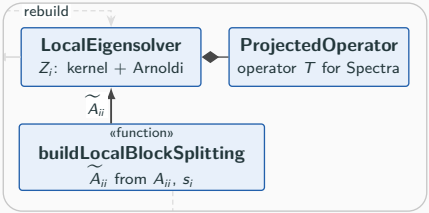
Zero row sums of A give $\widetilde{A}_{ii}\mathbf{1} = 0$ on floating subdomains: $\widetilde{A}_{ii}$ is singular, it cannot be factorized as it is.

6. Coarse space: the local eigensolver

`LocalEigensolver::compute`$(A_{ii}, \widetilde{A}_{ii}, D_i) \to Z_i$:

1. Factorize $\widetilde{A}_{ii} + \sigma I$, $\sigma = 10^{-13}\|\widetilde{A}_{ii}\|$.
2. **Kernel** K_i: block inverse iteration (multi-RHS solves) + SVD; left kernel by transposed solves if A is non-symmetric.
3. **Infinite eigenvalues**: $(L_i \cap K_i)^{\perp_{K_i}}$ computed exactly from an SVD of $B_i Q$, $B_i = D_i A_{ii} D_i$.
4. **Finite eigenvalues**: restarted Arnoldi (Spectra) on the matrix-free `ProjectedOperator`: $T = \Pi_K(\widetilde{A}_{ii} + \sigma I)^{-1}\Pi_R B_i \Pi_K$. one SpMV + one pair of triangular solves.

   Keep $|\lambda| \geq 1/\tau$: with $\texttt{nev} \geq 128$, start from $k = 32$ and enlarge while $|\lambda| \geq 1/\tau$: the cost follows the selected vectors.



6. Coarse space: assembly of A_{00} and coarse solvers

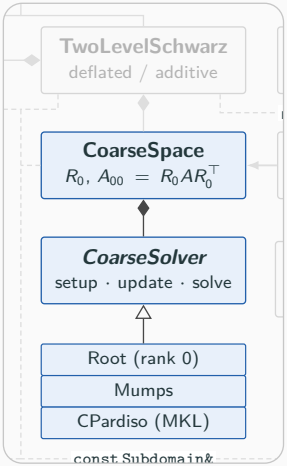
$V_i = \mathrm{orth}(D_i Z_i)$ by column-pivoted QR: same span, dependent vectors dropped, better conditioned A_{00}. Rank i owns its m_i coarse unknowns.

A_{00} from local products only, with $W = R_0^\top$ restricted to Ω_i (one exchange of sparse rows):

$$A_{00} = \sum_{i=1}^{N} W(\Omega_{I,i}, :)^\top \, A(\Omega_{I,i}, \Omega_i) \, W(\Omega_i, :)$$

Boolean PoU: restriction $V_i^\top R_i r$ and prolongation are communication-free.

`CoarseSolver` gets distributed triplets: `Root` (gather, factorize on rank 0), `Mumps`, `CPardiso` (MKL cluster solver).



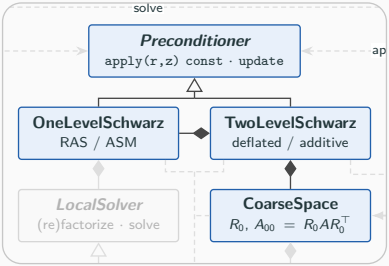
6. Coarse space: the two levels together

Setup (`TwoLevelSchwarz` constructor):

1. `OneLevelSchwarz`: factorization of A_{ii};
2. $\widetilde{A}_{ii} \to$ `LocalEigensolver` $\to Z_i$ (no communication);
3. failures reduced over all ranks before throwing (no rank left hanging in a collective);
4. `CoarseSpace`: V_i, A_{00}, coarse factorization.

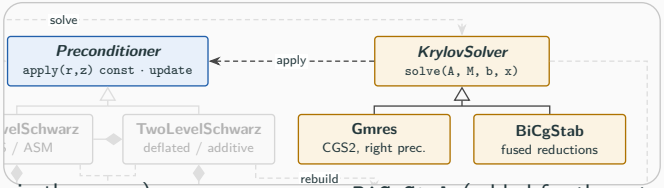
Application, transcription of the deflated formula:

```
coarse_->apply(r, x0_);    // x0 = R0^T A00^{-1} R0 r
A_.apply(x0_, w_);         // extra product of "deflated"
w_ = r - w_;
one_->apply(w_, z1_);      // M_RAS^{-1} (r - A x0)
z = x0_ + z1_;
```



Per application: 2 halo updates (local solve, extra SpMV) + the coarse solve.

7. Solvers: Krylov methods



`Gmres` (restarted, as in the paper)

- Right preconditioning: the residual is $b - Ax$, the stopping test of Section 4.
- CGS2: the j inner products of a pass in one `MPI_Allreduce`.
- Convergence accepted only if the true residual agrees (10%).

`BiCgStab` (added for the extras)

- The algorithm of OpenFOAM `PBiCGStab`, 4 fused reductions per step instead of 6.
- Stopping tests: $\|r\|_2/\|b\|_2$, or OpenFOAM's normalised residual (`relTol`, `tolerance`).
- Iteration = one application of M^{-1} in both: counts compare directly.

7. Solvers: verification

Every distributed component is compared with a sequential reference built from the global matrix; every check is reduced over the ranks. `make check`: every test on 1, 2 and 4 processes.

Test	What is compared
`test_subdomain`	overlap, halo, A_{ii}, lumping sums, partition of unity, distributed SpMV
`test_preconditioner`	sparse vs dense eigensolver and kernels; RAS/ASM $\times$ deflated/additive vs a reference assembled on one process; GMRES
`test_factorization`	Cholesky / LDL$^\top$ / LU chains, single precision, every coarse solver
`test_reuse`, `test_inexact`	refactorization vs fresh setup, reuse policy; AMG local solver (extras)
`test_bicgstab`, `test_stopping`	BiCGStab, OpenFOAM stopping test (extras)
`test_warmstart`, `test_bubbles`	warm start, bubble field shared with OpenFOAM (extras)

On G100 every job script builds its own binaries, runs the tests on the compute nodes and records the source hash and the loaded modules next to the results.

8. Reproduction of the paper: setting

Same as the paper

RAS + deflated, right-preconditioned GMRES(30), $\mathrm{rtol} = 10^{-8}$, $x_0 = 0$, random b.

SuiteSparse matrices of Table 1, convection–diffusion (4.1).

Different

CINECA Galileo100: 240 processes (paper: 256–4096).

METIS (ParMETIS), PARDISO (MUMPS), Spectra (SLEPc).

(4.1): upwind finite volumes (SUPG, P1/P2 elements).

Experiment	Outcome
Fig. 2, τ	reproduced: n_0 within 15% (26% at $\tau = 0.6$), iterations 9–29% lower
Table 1	n_0/N within 15% on 8 of 10 matrices, iterations ± 1 on 6
Fig. 4, strong scaling	iterations bounded in N: $16 \to 12$ (paper $28 \to 15$)
Table 2	2D and 3D with $\nu \geq 10^{-2}$; 3D, $\nu \leq 10^{-3}$: smaller n_0, more iterations
Failure	circuit5M_dc: setup > 20 min

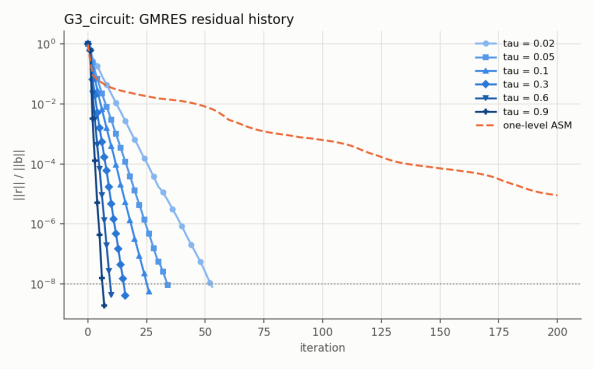
8. Reproduction of the paper: Table 1

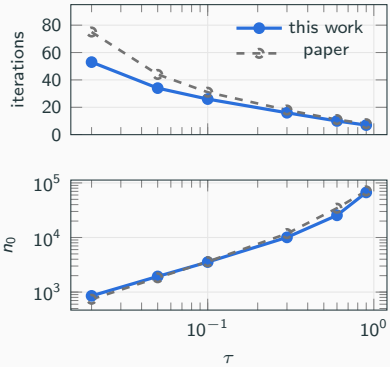
	Paper		This work		RAS
Matrix	its	n_0/N	its	n_0/N	its
finan512	6	10	6	22	6
nxp1	20	77	29	95	n.c.
CoupCons3D	20	113	21	98	23
tmt_unsym	6	129	5	128	n.c.
ecology2	6	133	5	130	n.c.
thermal2	26	157	5	139	n.c.
atmosmodj	7	298	14	286	79
G3_circuit	8	279	7	279	n.c.
Transport	9	300	8	300	n.c.
memchip	36	226	8	224	44

240 processes, nev= 300, $\tau = 0.9$. n.c.: not converged in 200 iterations.

- The paper states $\tau = 0.6$, but its G3_circuit row (8 its, $n_0 = 71\,385$) is the $\tau = 0.9$ point of its own Fig. 2 $\Rightarrow$ compared at $\tau = 0.9$.
- Wherever one-level RAS fails, the two-level method converges.
- Non-symmetric matrices: iterations also depend on the partition and on the random b (both differ between the codes).

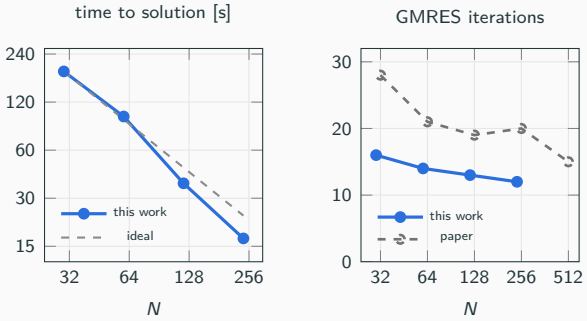
8. Reproduction of the paper: effect of τ (Fig. 2, G3_circuit)





Larger τ: more eigenvectors per subdomain, fewer iterations; one-level ASM does not reach 10^{-8}.

8. Reproduction of the paper: strong scaling (Fig. 4) and Table 2



3D (4.1), $\nu = 10^{-2}$, $8 \cdot 10^6$ unknowns, $\tau = 0.3$, nev$= 60$.

- Iterations bounded in N, as in the paper.
- Superlinear speed-up (11.1× for 8× the processes): setup is 96–97% of the time, and LU of A_{ii} and eigensolves grow faster than linearly with n_i.
- Table 2 (convection–diffusion): same trends; n_0 smaller than the paper (0.5–0.6× in 2D, $\approx$ 0.2× in 3D): FV stencils couple the overlap to fewer outside nodes $\Rightarrow$ less lumping.

9. Bubble flow: the problem

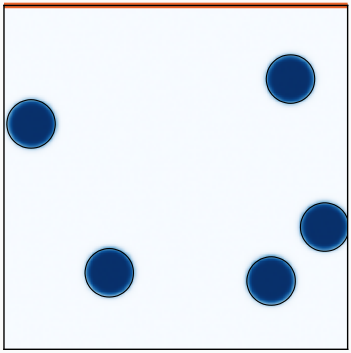
Pressure equation of an incompressible two-phase flow:

$$-\nabla \cdot (\beta \nabla p) = f, \qquad \beta = \frac{1}{\rho},$$

$$\rho = \alpha \rho_g + (1-\alpha)\rho_l, \qquad \frac{\rho_l}{\rho_g} = 10^3 \text{ (air in water)}.$$

Discretized with cell-centred finite volumes $\Longrightarrow$ A is SPD, diagonally dominant and an M-matrix.

Test case of the next slides: 128×128 cells, five bubbles of radius 0.07, $p = 0$ on the top side (`matgen bubbles2d`, same matrix as in the report).



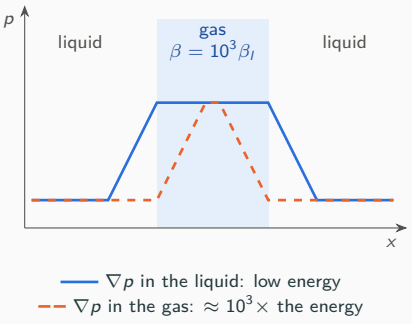
gas fraction α — $p = 0$

9. Bubble flow: why it is hard

Near-kernel modes. In the gas β is 10^3 times larger. A pressure field flat inside a bubble, varying only in the liquid, has an energy $\int \beta |\nabla p|^2$ about 10^3 times smaller than the same shape with its gradient in the gas: every bubble away from the Dirichlet boundary adds one such mode.

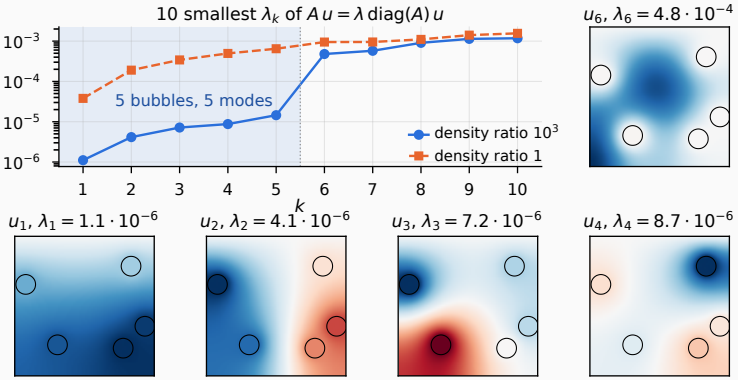
Hard for multigrid: the modes are not smooth, flat in the bubble and bent at the interface. GAMG agglomerates by face area, blind to β, and leaves them to the smoother.

Good application of the preconditioner: the local eigenproblems use only A and find these modes, whatever the bubble shape.



1D cut through a bubble

9. Bubble flow: near-kernel modes



Spectrum. Five eigenvalues lie 33–430× below λ_6, one per bubble. With uniform density the gap disappears.

Eigenvectors. $u_1, \ldots, u_5$ are flat on every bubble and bend only in the liquid. u_6, above the gap, vanishes in the gas.

Eigenpairs of $Au = \lambda\,\mathrm{diag}(A)\,u$ computed with SciPy (`scripts/bubble_modes.py`).

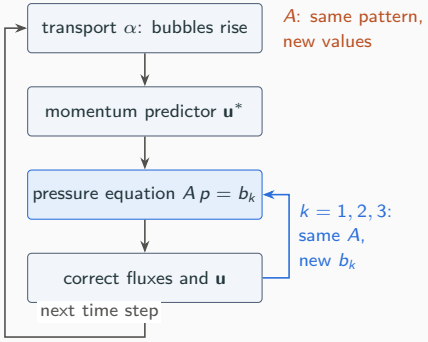
9. Bubble flow: the PISO time loop

PISO couples pressure and velocity in transient incompressible solvers. Every time step transports the interface, predicts the velocity and then corrects it two or three times.

Each corrector solves the pressure equation and makes the fluxes divergence-free. Its coefficient, $1/a_P$ of the momentum equation ($\approx \Delta t/\rho$), is frozen during the step: same matrix, new right-hand side.

From one step to the next the bubbles move: same sparsity pattern, new values near the interfaces.

In the tests `matgen` moves the bubbles by half a cell per step; three correctors, each from $x_0 = 0$ with a random b.



9. Bubble flow: what can be kept

Three update levels. Before every solve `Preconditioner::update(UpdateLevel)` receives one of three levels, and the setup is split by what each part depends on:

Depends on	Data	`Reuse`	`Refactorize`	`Rebuild`
graph of A	overlap, halo, patterns, D_i	kept	kept	kept
pattern of A_{ii}	symbolic factorization	kept	kept	redone
values of A	A_{ii}, $A(\Omega_l, \Omega_i)$, s_i; numerical LU of A_{ii}	kept	redone	redone
values of A and V	A_{00} and its factorization	kept	redone	redone
values of A	$\widetilde{A}_{ii}$, eigenproblems, V_i	kept	**old basis**	redone

Correctors 2 and 3 find the same values: `Subdomain::updateValues()` detects it with one reduction and the level becomes `Reuse`. New values cost one halo exchange and an indexed copy.

One decision for all ranks: `agree()` takes the highest level over the ranks. The measured times differ slightly from rank to rank, and a rank rebuilding alone would deadlock in the collectives of the coarse level.

9. Bubble flow: when to rebuild (P1)

Aging. A refactorization keeps A_{ii} and A_{00} exact, but the basis V_i was computed for bubbles that have moved: the iterations grow.

Waste. With d decades of residual reduction in k iterations, $q = k/d$, and q_{ref} its value right after a rebuild, every solve wastes $(k - q_{\mathrm{ref}}\, d)\, t_{\mathrm{it}}$ seconds because of the old basis.

Break-even. `ReusePolicy` rebuilds when the waste since the last rebuild reaches the extra cost of a rebuild over a refactorization, both measured in the run (lower medians). This rent-or-buy rule never spends more than twice the best choice in hindsight, with no threshold to tune.

Problem	`off`	`auto`	gain
bubbles 128^3	3.47 s	0.611 s	**5.7**×
bubbles 192^3	16.0 s	2.62 s	**6.1**×
conv.-diff. 100^3	1.25 s	0.169 s	**7.4**×

Cost per solve, setup included, 240 processes; bubbles with $\rho_l/\rho_g = 10^3$, convection–diffusion non-symmetric. `off`: new setup before every solve; `auto`: refactorize at every step, rebuild at break-even.



9. Bubble flow: setup reuse on G100 (P1)



Iterations. One-level RAS needs about 800. With `refactor` (basis never rebuilt) they grow from 12 to 47, then settle around 30; `auto` rebuilds every $\sim$9 steps, a saw-tooth from 12 to 31–39.

Cost. Both reuse modes are about 6$\times$ cheaper than `off`. Here `auto` and `refactor` end together (0.611 and 0.615 s per solve): the rule does not need to know in advance whether rebuilding pays.

Bubbles 128^3, $\rho_l/\rho_g = 10^3$, 240 processes, GMRES(30) to 10^{-8}, 3 correctors per step; dots: rebuilds chosen by `auto`. The jump near solve 160 is a timing burst, not a rebuild.

9. Bubble flow: cheaper solves (P2, P5, P7)

Method. Each phase starts from a measured bottleneck, changes one thing, passes the unit tests and is measured in a campaign on G100 (`test/*/RESULTS.md`).

Phase	Change	Cost per solve	
P2	Cholesky instead of LU for A_{ii}, $\widetilde{A}_{ii} + \sigma I$ and A_{00}	-10% **to** -18%	same iterations
P5	at most $\mathtt{nev} = 8$ coarse vectors per subdomain	-52% **to** -80%	rebuilds 4–6$\times$ cheaper
P7	one AMG V-cycle instead of exact factors of A_{ii}	-26% **to** -56%	same iterations after a rebuild

Why P7 is allowed. Schwarz theory admits any local solver spectrally equivalent to A_{ii}^{-1}. One V-cycle of smoothed aggregation from $x = 0$ is a fixed linear operator, so plain GMRES stays valid. It reads about $1.5\,\mathrm{nnz}(A_{ii})$ a few times, while a 3D Cholesky factor has 20–35$\times$ the nonzeros of A_{ii}.

Measured and dropped: incomplete Cholesky (2.4–8.7$\times$ the iterations: not spectrally equivalent), other AMG smoothers, reusing the AMG hierarchy across refactorizations, single-precision factors as a default, warm-started eigenvectors.

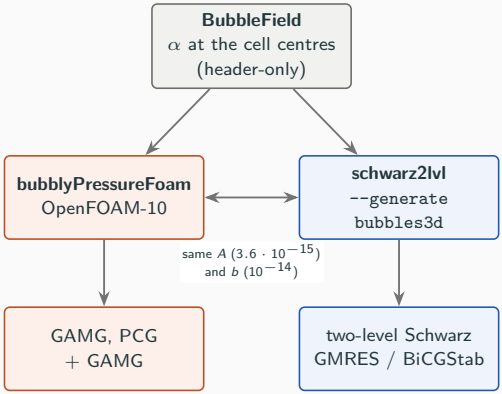
10. Performance vs GAMG: setting

Same problem on both sides. The OpenFOAM-10 solver `bubblyPressureFoam` assembles the same equation with the same header-only `BubbleField`: the matrices agree to $3.6 \cdot 10^{-15}$ (relative), checked in every job.

GAMG runs with the OpenFOAM defaults, `faceAreaPair` agglomeration and DIC smoother, the fastest variant tested.

Schwarz runs with $\tau = 0.3$, `nev` $= 8$, AMG local solves and reuse `auto`.

Meshes of 192^3, 256^3 and 320^3 cells (7.1–32.8 M) on 240, 336 and 432 processes. The cost of a solve includes setup, updates and iterations.



10. Performance vs GAMG: every solve from zero

Cost per solve [s], Schwarz / GAMG, `relTol` 10^{-2} of the OpenFOAM residual, 16 steps $\times$ 3 correctors:

	240 proc.	336 proc.	432 proc.
192^3	0.316 / 0.567	0.258 / 0.570	0.206 / 0.563
	1.79$\times$	**2.21**$\times$	**2.73**$\times$
256^3	0.986 / 1.99	0.724 / 1.65	0.573 / 1.48
	2.02$\times$	**2.28**$\times$	**2.59**$\times$
320^3	2.38 / 4.74	1.80 / 3.39	1.12 / 2.74
	1.99$\times$	**1.88**$\times$	**2.44**$\times$

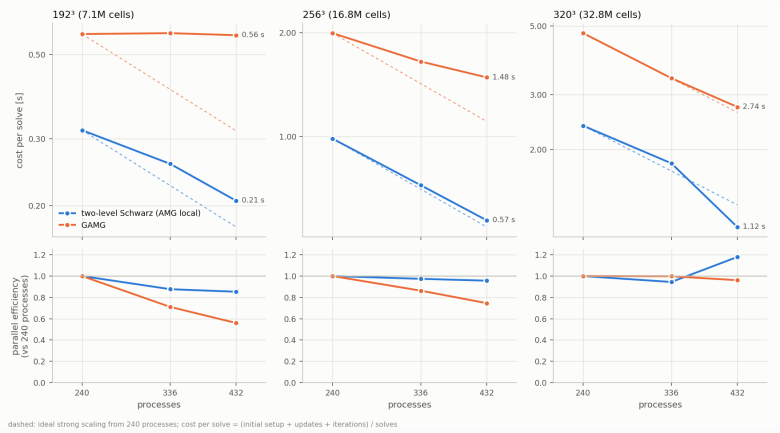
Schwarz with GMRES(30); GAMG with DIC. Ratio GAMG / Schwarz, >1: Schwarz faster.

Always faster: the gain holds on all 9 configurations, 1.8–2.7$\times$.

At a tight tolerance, $\|r\|_2/\|b\|_2 \leq 10^{-8}$ on 240 processes, it grows to 2.0$\times$ at 192^3 and 4.3$\times$ at 256^3: GAMG goes from 139 to 271 V-cycles, Schwarz needs 51 iterations at both sizes.

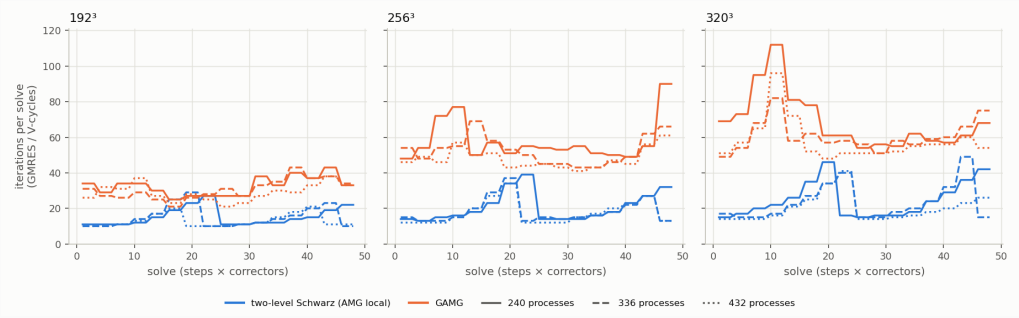
Setup and updates are about 30% of the Schwarz cost at 10^{-2}.

10. Performance vs GAMG: strong scaling



Parallel efficiency from 240 to 432 processes: Schwarz **0.85 / 0.96 / 1.18**, GAMG 0.56 / 0.75 / 0.96 (192^3 / 256^3 / 320^3).

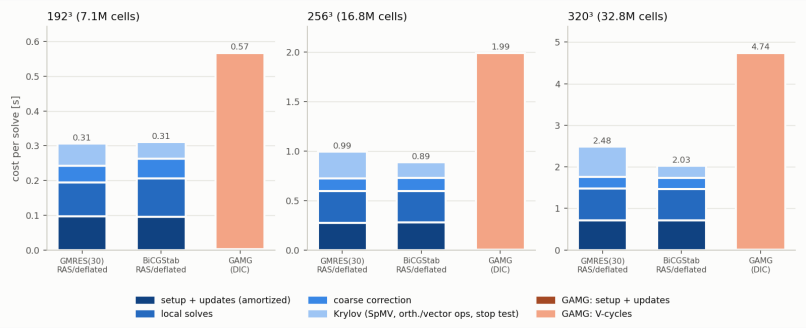
10. Performance vs GAMG: where the gain comes from



Same cost per iteration. One Schwarz iteration costs about one GAMG V-cycle: 34.8 vs 34.5 ms at 256^3, 68.9 vs 68.8 ms at 320^3. The gain is entirely in the iterations, 2.0–2.9× fewer.

Iterations. From 192^3 to 320^3 GAMG goes from 32 to 69 V-cycles, Schwarz after a rebuild from 11 to 15. The saw-tooth is the coarse space that ages as the bubbles move, until the break-even rebuild.

10. Performance vs GAMG: Krylov method



GMRES pays CGS2 and the OpenFOAM stopping test with $O(j)$ passes over the basis, and j grows with the mesh.

BiCGStab needs 1.04–1.20× the applications, each 0.72–0.85× as expensive: 0.89 and 0.82× the cost of GMRES at 256^3 and 320^3 (1.02× at 192^3). It is OpenFOAM's `PBiCGStab`, the natural host of a port.

10. Performance vs GAMG: a PIMPLE-like time loop

Closer to an OpenFOAM run. The right-hand side is the p_{rgh} source of `interFoam`, on the interfaces and with zero sum over every bubble. Every solve starts from the previous solution (warm start); the correctors stop at `relTol` 0.05, the last one at `tolerance` 10^{-7}.

Decision rule, fixed before the run

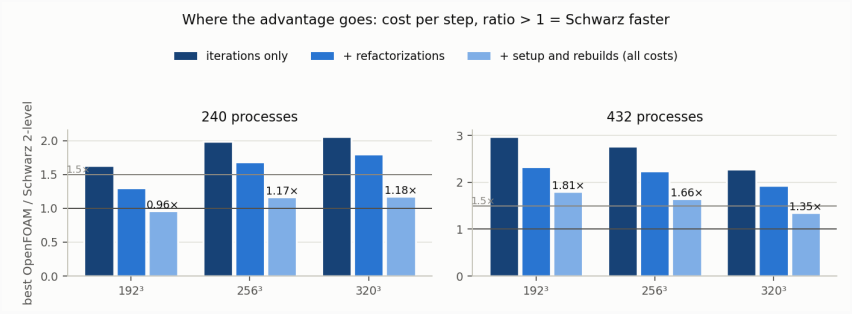
Port only if $\geq 1.5\times$ faster than the best OpenFOAM variant at both process counts.

Mesh	Proc.	Schwarz	GAMG	ratio
192^3	240	0.605	0.581	0.96×
256^3	240	1.68	1.97	1.17×
320^3	240	4.30	5.08	1.18×
192^3	432	0.358	0.647	**1.81**×
256^3	432	0.911	1.51	**1.66**×
320^3	432	2.06	2.79	1.35×

Cost per time step [s]: 3 correctors, setup and rebuilds included, 24 steps.

Outcome. Faster than GAMG in 5 of 6 configurations, but the $1.5\times$ threshold is reached only at 432 processes on the two smaller meshes: a port is not justified as it stands.

10. Performance vs GAMG: reading the PIMPLE-like result



The gain moves to the last corrector, 2.0–4.1×. On the first two, 1.3 decades from a good guess, GAMG is 1.2–1.7× cheaper: little of the error lies in the bubble modes.

Setup and rebuilds take 39–43% of the Schwarz cost. The iterations became cheap, the setup did not.

10. Performance vs GAMG: why the gap narrows in the time loop

GAMG cannot reduce the error along the bubble modes u_k, flat on each bubble and invisible to a coarsening blind to β.

The error depends on b and x_0. From $x_0 = 0$:

$$e_0 = x = \sum_k \frac{u_k^\top b}{\lambda_k} u_k,$$

the tiny λ_k amplify the bubble modes, unless $u_k^\top b \approx 0$.

	from zero	time loop
b	α on the bubbles	p_{rgh} source
$u_k^\top b$	$\neq 0$	≈ 0
x_0	0	previous x
energy in $u_1, \ldots, u_{10}$	**99%**	$<$ **0.1%**
GAMG / Schwarz	**1.8–2.7**$\times$	0.96–1.81$\times$

In the time loop the p_{rgh} source sums to zero over every bubble and the warm start already carries the bubble levels: the test from zero was the worst case for GAMG.

The coarse space is still needed: one-level RAS costs 2.9–5.7$\times$ GAMG (420–700 applications on the last corrector).

11. Conclusions

Implementation

Fully distributed C++17/MPI library: its only input is `LocalSystem`, the rows that an OpenFOAM processor owns.

Run-time strategies (local and coarse solvers, Krylov method, reuse) sit behind small interfaces and factories; templates only where just the types vary.

Section 4 of the paper is reproduced on 240 processes, and the distributed parts are checked against sequential references.

Bubbly Flows

Setup reuse, a small coarse space and AMG local solves make a solve about 30× cheaper ($16.0 \rightarrow 0.51$ s at 192^3 on 240 processes).

From zero the method is 1.8–2.7× faster than GAMG and scales better: the gain is algorithmic, it comes from the iterations.

In a PIMPLE-like loop the ratio is 0.96–1.81× per step, and the setup is about 40% of the cost.

Next step

A coarse space that follows the bubbles at the cost of a refactorization: keep the eigenvectors of the last rebuild and add indicator vectors of the gas regions of each subdomain ($O(n_i)$). A port would be an `lduMatrix::preconditioner` inside `PBiCGStab`, with the state in a `MeshObject`.